



IDG2100 2026 Full-Stack Web Development: Exam Project Description

NOTE: this document is a draft and minor fixes and clarifications may be added to it in the days following its initial distribution.

Context & Exam Structure

The exam in IDG2100 2026 is an oral defense of a group programming project. Teams are intended to have three members and were formed as a part of activities for the mandatory assignment #4 (Oblig 4).

The project is a continuation of the work done individually by students in three previous mandatory assignments (Obligs 1, 2 and 3). You **are allowed** to use previously delivered code and repositories (for Obligs 1, 2 and 3) as starter code for this project, but this has to be clearly acknowledge in the [NOTES.md](#). The decision on whose repository to rely and how to rework it is up to the teammates: a member can contribute between zero to all three repos.

The teams themselves structure and distribute the workload among teammates, but everybody has to contribute to programming, everybody has to know all parts of the project and understand all code, and everybody **can be asked about any part** of the project during the exam, besides general programming or theoretical questions.

Before the oral defense, the project code has to be delivered in both GitHub Classrooms and Blackboard. The details on defense dates and place will be posted on Blackboard shortly before the exam dates (June 3rd-5th). Additional detail regarding the project and its delivery may be posted on Blackboard.

Although you may utilise snippets of code from tutorials and official documentation, you must clearly acknowledge the sources in the comments of your code. Plagiarism or cheating will be deemed to have taken place if the submitted code shows substantial similarities to other students' assignments or projects found online. In such cases, the matter will be reported to the NTNU appeals committee for further examination. If you have any doubts regarding the

use of materials for your project, please reach out to the instructor for clarification.

The code exhibiting signs of being AI-generated may result in additional questioning during the defense. Your understanding of the code and principles behind it will be prioritized during the exam over feature completeness.

Exam Goals

This exam aims to prove and demonstrate that you have achieved the learning outcomes, knowledge, skills and general competence described in [the course description](#).

You are specifically expected to demonstrate that:

- You understand the MERN technology stack and can build a full-stack modern web application using it
- You can design a REST API following best practices for API design, Express.js server architecture, and MongoDB database architecture
- You can partition a Web UI in suitable components and implement them using React (functional components)
- You understand and can follow best practices for designing and implementing React applications
- You understand security concerns related to MERN apps and can implement a suitable authentication and authorization system
- You can use modern developer tools such as task runners, bundlers, and pre-processors.

Project Description: Spanish Poker Dice Platform

Scenario

Your team has been called upon to take over the different uncoordinated and unfinished attempts to implement the parts of the Spanish Poker Dice Platform. You are to review existing, available code repositories and decide what to keep, what to rework, and what to implement anew. You are expected to deliver a fully functioning Web app, with a robust and secure backend, responsive and user-friendly frontend, and local database. Populate the database with dummy data before showcasing the project to the customer. If something is unfinished before the showcase, you will have to justify it to the customer, and it may result in

penalties for your team, depending on the scale of what is unfinished.

Features to Implement

The features that were described in the mandatory assignment 3 (Oblig 3) have to be implemented, including:

- Static pages: privacy policy, about us, about Spanish Poker Dice, and terms and conditions.
- Homepage showing a brief platform description, game lobby preview, functionality to create a game, and top N games being played (or most recent games if there aren't enough on-going games).
- Lobby page showing the games that the user can join, with pagination implemented.
- Individual game page showing the game board, game description (meta info), and comments in a sidebar.
- Log in, registration and forgot-password pages.
- User profile page, showing the user profile image, username, email (only shown to the user themselves or admins), and about me description - all of these editable by the user. The user's password should also be changeable. The user stats and most recent games should also be shown. The recent games should be paginated, with more games loadable on request. The stats include the user's Elo rating in the three time controls, total number of played games, and number of losses/wins in the last month.

The header - besides the main nav menu, login/logout controls, and greeting message (with a profile pic preview) - have an expandable item to customize page appearance: site theme (dark/light or similar), sound on/off, board background color, and # of games to show in the lobby preview.

If you are unsure about some of the above features, consult the description of the mandatory assignment 3.

The new features include:

- The **upcoming-tournament overview** on the homepage: a list of 5 upcoming tournaments that the user can either join or spectate. Show the sufficient detail for each tournament, e.g., title, date/time, game variant, number of rounds, and how many players can join and/or have signed up. Clicking on a tournament should redirect the user to the

individual tournament page.

- **Tournament list page** (reachable from the main menu and/or from the button/link under the tournament overview component above), which should provide more detail than the tournament overview component, including tournament rules, status (upcoming, ongoing, finished), author, trophy description and image. The list should include pagination (a load-more button). The page should show separately upcoming/ongoing tournaments and past tournaments (same info, but separate components).
- **Individual tournament page** should provide even more detail than the tournament list page, including the full description (not just a title), full-size trophy image, list of joined (and awaiting the tournament start) players, current standings (for ongoing tournaments), tournament comments, and a button to join/leave the tournament (if available to the user). Tournament rules include a game variant, if it's open to specific Elo ranges, and size of buy-in (the number of points the player need to have to enter). *Platform admins* should see a button to delete a tournament, a button to cancel a tournament (users can still see it, but can't join), and a button/link to the page to edit the tournament. Clicking the edit button should redirect to the admin page for tournament creation (see below) with fields pre-filled with current values. Tournament-related logic includes:
 - Users can only join a tournament before it starts.
 - Users can leave a tournament at any point.
 - Players (the users that have joined the tournament) are automatically re-directed from the tournament page to a game page and back (upon game completion). Other users are instead shown the list of on-going games in the tournament. They click on any game and view it.
 - The tournament page shows a countdown till the next round of games starts.
 - Only one type of tournaments has to be implemented^[1], random-pairing round-based tournaments: a tournament has a fixed number of rounds (the rounds here are a different concept from the in-game rounds^[2]) and players are assigned into games randomly. The process is repeated for each round. The ranking and tournament winner are determined based on the number of points obtained in it.
 - Players may be given some extra points (see the point on points below) for winning a tournament.
- **Sorting:** the lists of tournaments (tournament list page) should be sortable (by date, title, and # of players) and searchable (by title, simple string partial match after at least 3 characters entered).
- **Filtering:** the list of game (lobby page) should allow for filtering out game variants (all 3

dimension: straights allow, 3/5/7 rounds, and the time control) that the user is not interested in.

- The individual game page should allow for the user to **leave the game** before its start.
- Add **points** to the game: each week a player gets 100 points that use to bet in games. The number of points is shown on the profile page. Points are lost and gained in games.
- Expand available **game variants** to also account for the number of players (2/3/5 players) and size of buy-in (1/10/50 points). Change time controls - instead of having 3/10/30 sec per round, have 10/30/90 seconds in total, for all rounds.
- **Playing the game** functionality:
 - Implement a game board component (and all necessary nested components, e.g., Player and Die) using [Web Components](#).
 - Wrap the game board component in a React component if necessary. Ensure that the state of the game is restored on page reload/navigation.
 - Rolls are generated on the backend and sent to the frontend. The frontend only communicates held dice. The backend estimates the winner based on the rolls and holds, and enforces time if needed.
 - The info about rolls and holds is communicated via [Web Sockets](#) (or any other technology suitable for low-latency message exchange on the web). Other's rolls and holds are communicated (and rendered) to all players as quickly as possible.
 - The number of players should be between 2 and 5. The game starts automatically once the required number of player clicked "join" the game.
 - The rolls of others are not revealed to the player. This allows for bluffing. Only the backend knows all rolls and holds before the dice are revealed at the end of a round.
 - Implement betting: players look at their dice and bet an amount of their points (aka, put these points in the pot). Others look and their own dice and either match the amount, fold (already bet points are not returned and remain in the pot), or raise (increase the amount of bet points further). The dice are revealed when everyone either folder or have matched the highest bet amount. The winner collects the points from the pot (draws lead to splitting points). After the end of all rounds (aka, game end), the amounts in individual player's stacks are added back to the points the user has on their profile.
 - A player needs to have enough points on their profile for a "buy-in" - the amount that is reserved from their profile and what they start the game with - to join a game.
 - Elo is re-estimated at the end of the game. You will need to adapt the algorithm to having more than 2 players: run the original Elo re-estimation algorithms for each pair

of players; a player "loses" pairings against others who ended up with more points and "wins" the pairings against those with fewer points.

- If a player runs out of time (or abandons the game), the dice are still rolled for them, but they can't hold any or re-roll, and always match the amount in the pot.
- If you are unsure about the rules of Spanish Poker Dice, consult online sources (e.g., [encyclopedias](#)).
- If the user enabled sounds (using the settings button in the header), sounds are played for round starts and ends, rolls, holds, and game end.
- Implement actual **authentication and authorization**, using refresh and access tokens and relevant technologies (cookies, JWT, hashing and salting). New users have to have their email verified before they can play games: send an email with a link with a verification code to the user's email. The user clicks the link and is shown a message (can be a dedicated page in your frontend), either that verification passed and they can login now or that the code was not okay (either invalid or expired), also offering to re-send the code. The code expires after 15 minutes.
- Implement a **not found** page.
- Implement **comments using Web Sockets** so that new comments are displayed to all users without page reloads.
- Show **platform activity** on the homepage: the number of active players and played games in the last week, and number of available games right now.
- Implement four **admin pages**: dashboard, user administration, comment administration, and tournament creation.
 - The dashboard should be accessible from an extra top-menu item shown only to admins. It should display platform activity: # of new profiles in the last week, security incidents, and the info shown to the regular users on the homepage as platform activity (see above). The dashboard should also display the links to the other three admin pages. The two types of security incidents to display are:
 - Somebody hitting API limits, e.g., more than 100 requests per minute. Record and display their IPs and user agents and the time of the incidents.
 - Someone's IP changing^[3]: when the user requests an access token (JWT), you record their IP in the token and compare that IP against the IP of incoming requests with the token. If there is a mismatch, you record the incident and respond with 401, triggering the client to re-obtain a new access token.
 - *User administration* page would allow for searching through, listing and banning user profiles or making some users admins.

- *Comment administration* page should show recent comments and allow for comment deletion.
- *Tournament creation* page should allow for tournament creation (a form and a submit button). Creating one redirects to that individual tournament page.
- All admin pages should show no footer and no other controls in the header besides the logo (as the link to homepage) and links to the admin pages themselves.
- **Anonymous** user can no longer play games, but they can still spectate. Trying to join a game/tournament should either show a message to them or redirect them to the login page.
- The focus in **matchmaking** shifts from the queue (see Oblig 2) to rooms in which players wait for the game to begin. A room corresponds to the individual game page on the frontend.

Technical Requirements

The project has to be implemented using React and Web Components on the frontend and Express.js with MongoDB/Mongoose on the backend. The solution has to be deployable locally and include a seeding script with enough DB inputs to showcase the project. The solution has to be robust and secure enough to resist common forms of platform abuse.

1. You are welcome to implement other types of tournaments. ↩
2. You can re-name in-game rounds into something else, e.g., steps, cycles, or phases, if you wish to differentiate them further from the tournament rounds. ↩
3. You would not do such IP comparisons in a real project (you would instead look for suspicious signals for refresh token, e.g., you would infer a country from the IP and estimate if the travel was possible in the time since the last refresh token). This is simply an exercise in logging security incidents. ↩